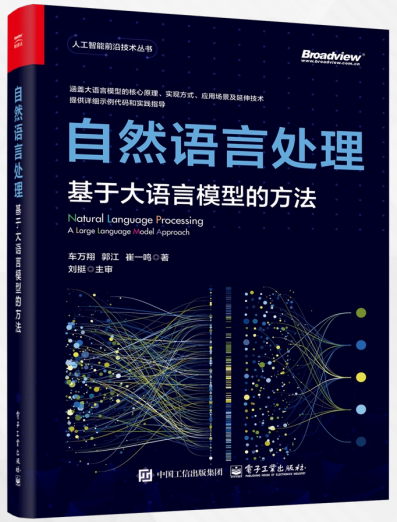


预训练语言模型

车万翔

社会计算与交互机器人研究中心（HIT-SCIR）
哈尔滨工业大学

本讲内容与学习目标

课堂内容	学习目标
概述与三种结构	区分注意力可见范围与预训练目标
Decoder-only：GPT系列	理解因果语言模型、精调与语境学习
Encoder-only：BERT及其改进	掌握MLM、NSP及RoBERTa等改进
Encoder-Decoder：T5与BART	理解文本到文本与去噪预训练
下游任务应用	为分类、阅读理解、序列标注搭建模型
研究进展与练习	了解现代编码器、架构之争与扩散语言模型

从词向量到预训练语言模型

2013	2018.2	2018.6	2018.10	2019	2020
Word2vec 静态词向量，作为下游模型的输入特征	**ELMo** 双向LSTM语言模型，提供上下文相关特征	**GPT** Transformer解码器预训练，再精调整个模型	**BERT** Transformer编码器，掩码语言模型实现深层双向	**GPT-2、T5等** RoBERTa、BART等：更多数据、更大模型、更统一的任务形式	**GPT-3、ELECTRA** 语境学习；更高效的预训练任务

范式转变：特征 + 任务模型 → 预训练 + 精调 → 提示 + 语境学习

本章聚焦2018—2020年的预训练语言模型；大语言模型的微调、适配与应用见后续章节。

三类预训练目标

预训练目标	模型输入	预测目标	代表模型
自回归语言模型	我 喜欢 读	下一个词：书	GPT系列
掩码语言模型	我 [MASK] 读 书	被遮住的词：喜欢	BERT系列
去噪 / 片段还原	Thank you <X> me to your party <Y> week	<X> for inviting <Y> last <Z>	T5、BART

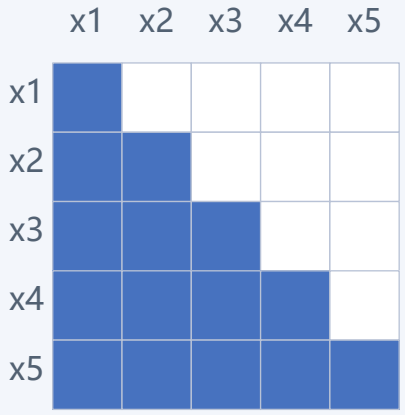
共同点 标签来自文本本身，不需要人工标注，即自监督学习

不同点 可见的上下文范围、每个样本的预测位置数量、能否直接生成文本

<X>、<Y>、<Z>是T5的哨兵词元，分别标记被删除片段的位置。

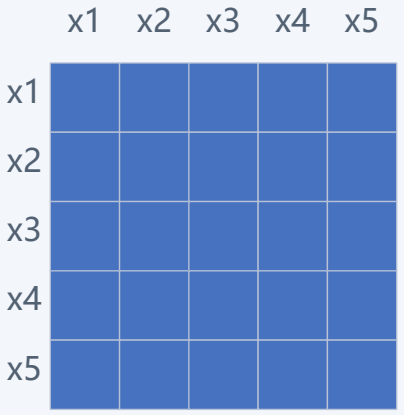
三种结构：注意力能看到哪些位置

Decoder-only



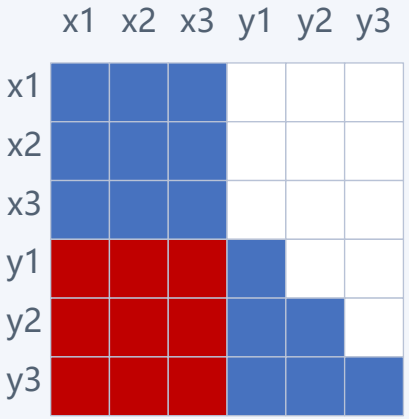
代表： GPT系列
擅长： 文本生成、续写

Encoder-only



代表： BERT、RoBERTa
擅长： 分类、抽取、标注

Encoder-Decoder

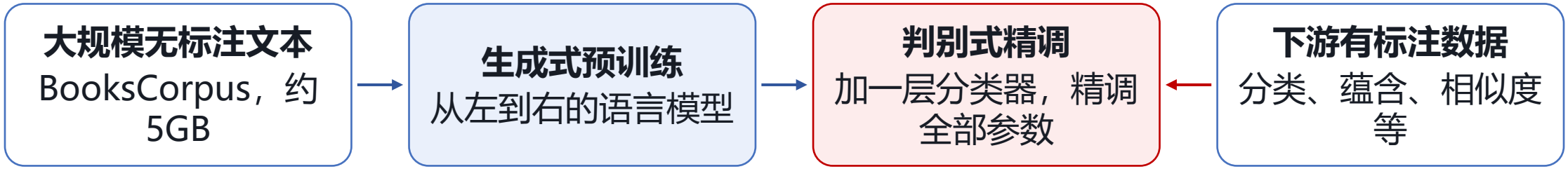


代表： T5、BART
擅长： 翻译、摘要

着色格子 = 可见。行是当前位置，列是它能读取的位置。

红色格子是Encoder-Decoder中解码器对编码器输出的交叉注意力。

GPT：生成式预训练 + 判别式精调



模型规模 12层Transformer解码器，768维，约1.17亿参数，上下文长度512

关键变化 精调整个预训练模型，下游只需很简单的输出层

历史意义 开启“预训练 + 精调”的自然语言处理新范式

Radford等，Improving Language Understanding by Generative Pre-Training，OpenAI，2018。

GPT的预训练：从左到右的语言模型

$$\mathcal{L}^{PT}(x) = \Sigma_i \log P(x_i \mid x_{i-k} \dots x_{i-1}; \theta)$$

$$h^{[0]} = e_{x'} W^e + W^p$$

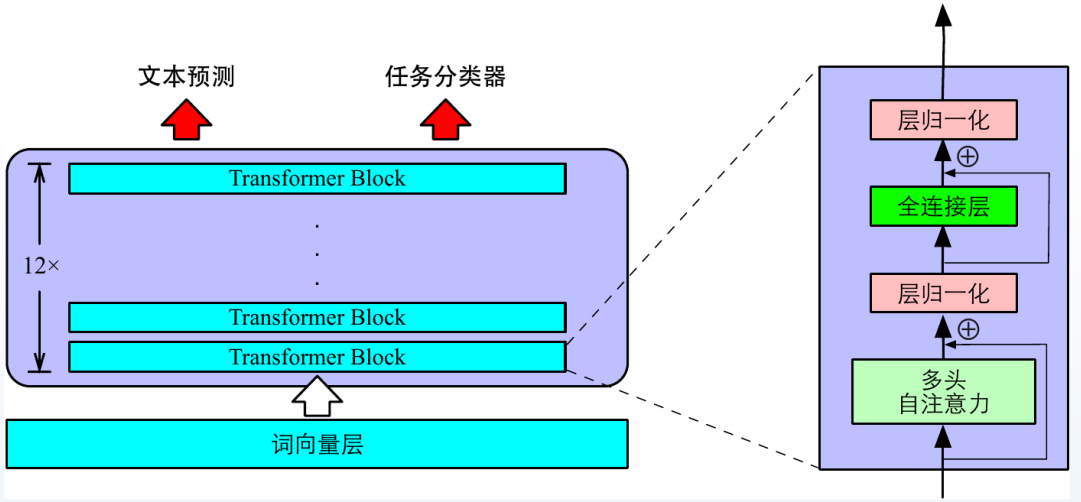
$$h^{[l]} = \text{TransformerBlock}(h^{[l-1]})$$

$$P(x) = \text{Softmax}(h^{[L]} W^{eT})$$

W^e　词向量矩阵，输入与输出共享

W^p　可学习的位置向量

k　上下文窗口长度，GPT为512



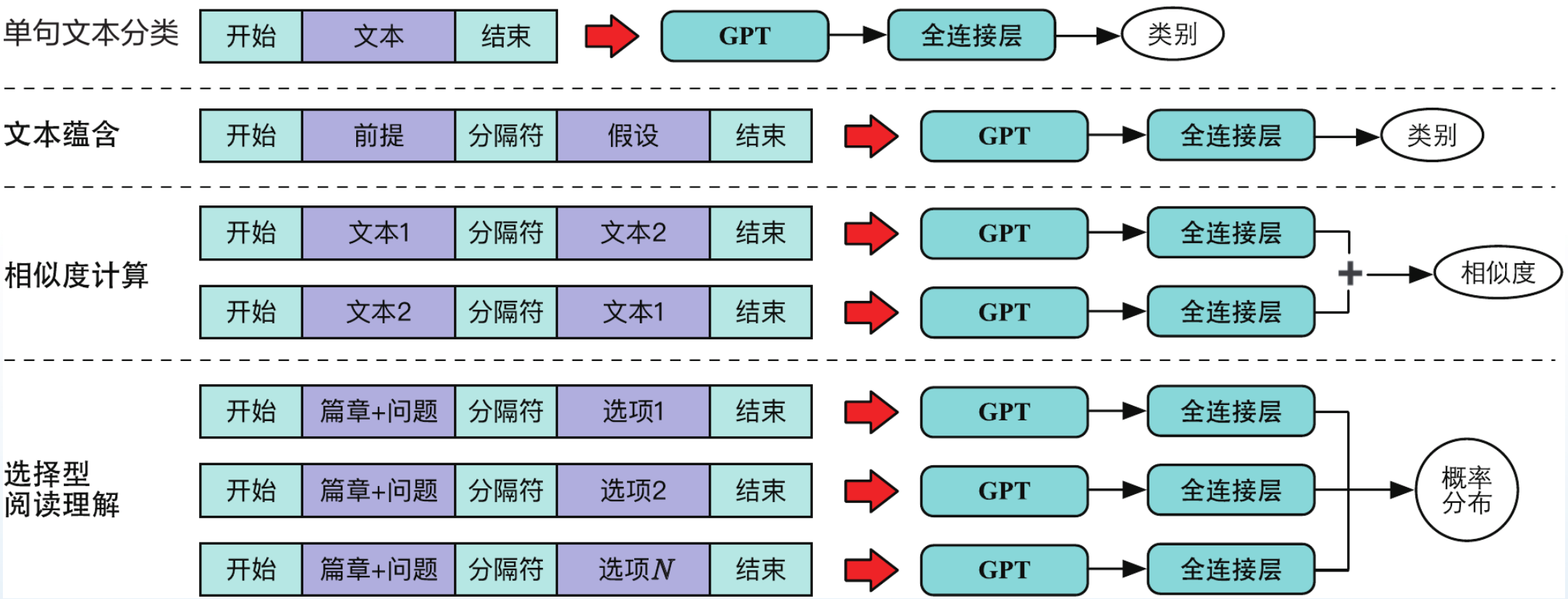
自注意力使用因果掩码，保证预测x_i时只用到它左侧的词。

PyTorch实现：因果掩码

```
import torch
T = 4
scores = torch.randn(T, T)                          # 注意力打分 [T, T]
mask = torch.triu(torch.ones(T, T, dtype=torch.bool), diagonal=1)
scores = scores.masked_fill(mask, float('-inf'))
attn = scores.softmax(dim=-1)                       # 每行只在当前及之前位置上归一化
print(mask.int())
# tensor([[0, 1, 1, 1],
#         [0, 0, 1, 1],
#         [0, 0, 0, 1],
#         [0, 0, 0, 0]])
print(attn[0])                                      # tensor([1., 0., 0., 0.])
```

1表示被遮住的“未来”位置。解码器每一层的自注意力都使用这张掩码。

用输入格式适配不同任务



模型结构不变，只改变输入的拼接方式和输出层。

<s>：开始；$：分隔；<e>：结束。相似度任务两种顺序各算一次；选择题每个选项单独打分。

GPT的精调：任务损失 + 语言模型损失

$$P(y \mid x_1 \dots x_n) = \text{Softmax}(h_n^{[L]} W^y)$$

$$\mathcal{L}^{FT}(\mathcal{C}) = \Sigma_{(x,y)} \log P(y \mid x_1 \dots x_n)$$

$$\mathcal{L}(\mathcal{C}) = \mathcal{L}^{FT}(\mathcal{C}) + \lambda \mathcal{L}^{PT}(\mathcal{C})$$

λ	含义
0	只优化下游任务损失
0.5	GPT论文的设置：以任务为主，保留部分语言模型目标
1	两项损失同等重要

目的：缓解灾难性遗忘，提高精调后的通用性与收敛速度。最大化上述对数似然。

GPT-2：更大的模型，零样本完成任务

比较项目	GPT	GPT-2
模型大小	117M	117M、345M、762M、1542M
训练数据	BooksCorpus（约5GB）	WebText（约40GB）
词表大小	40,478	50,257
上下文长度	512	1,024
训练方式	无监督预训练 + 有监督精调	仅无监督预训练，零样本推理

结构调整 层归一化移到每个子层的输入（Pre-LN），训练更稳定

主要发现 语言模型本身可以作为无监督的多任务学习器

Radford等，Language Models are Unsupervised Multitask Learners，OpenAI，2019。

GPT-3：语境学习

```
Translate English to French:
sea otter => loutre de mer
plush girafe => girafe peluche
cheese =>
```

参数量 1750亿

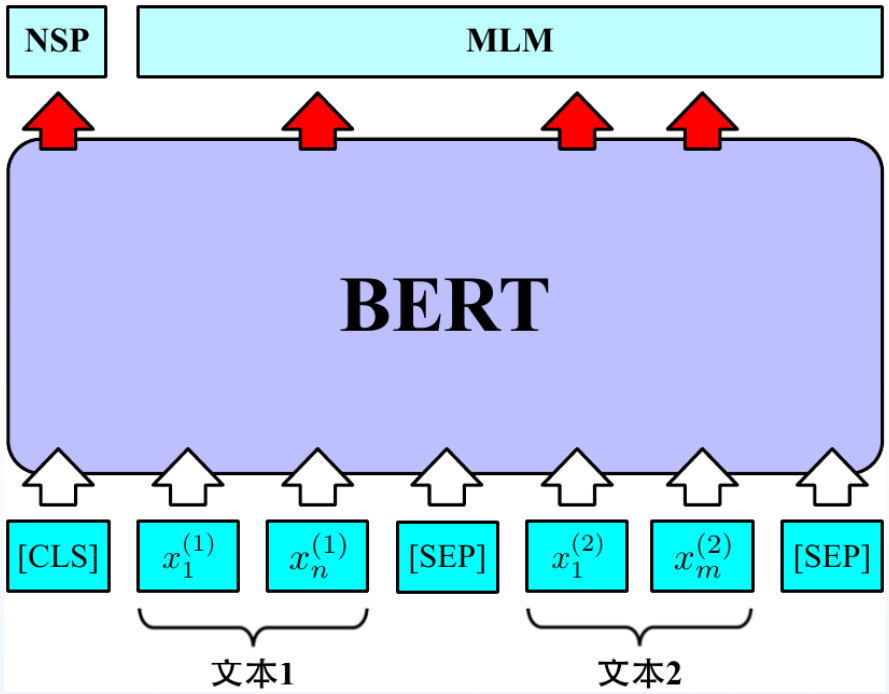
上下文 2048个词元

使用方式 不更新参数，只在输入中给出示例



上：任务描述 + 示例 + 待回答的问题。下：语言模型训练中的“内循环”与“外循环”。

BERT：深层双向Transformer编码器



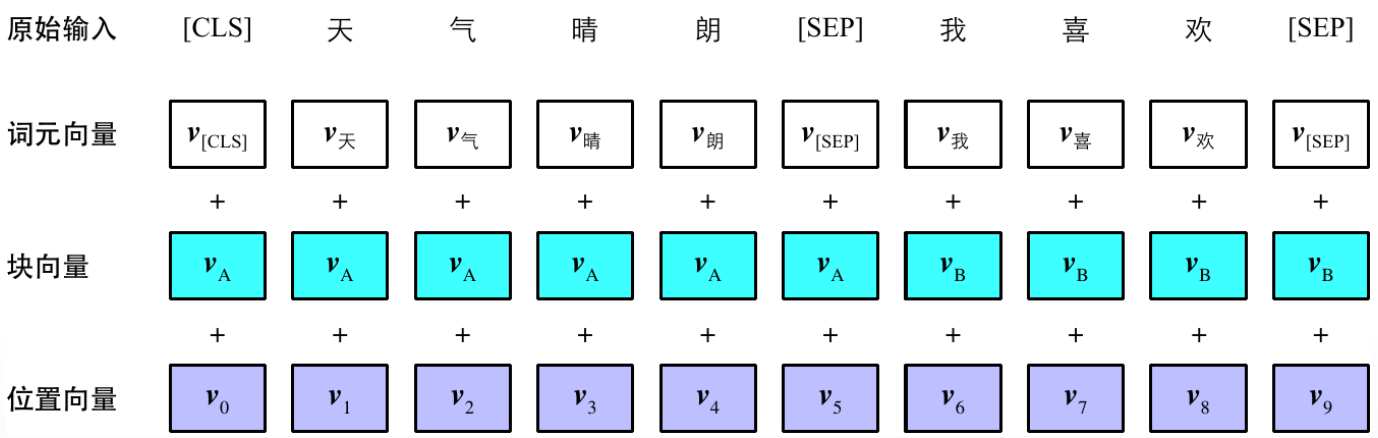
配置	BERT-base	BERT-large
层数 L	12	24
隐含层维度 H	768	1024
注意力头 A	12	16
参数量	约1.1亿	约3.4亿

数据 BooksCorpus + 英文维基百科，约33亿词（16GB）

任务 掩码语言模型（MLM）+ 下一个句子预测（NSP）

Devlin等，BERT，NAACL 2019。

BERT的输入表示

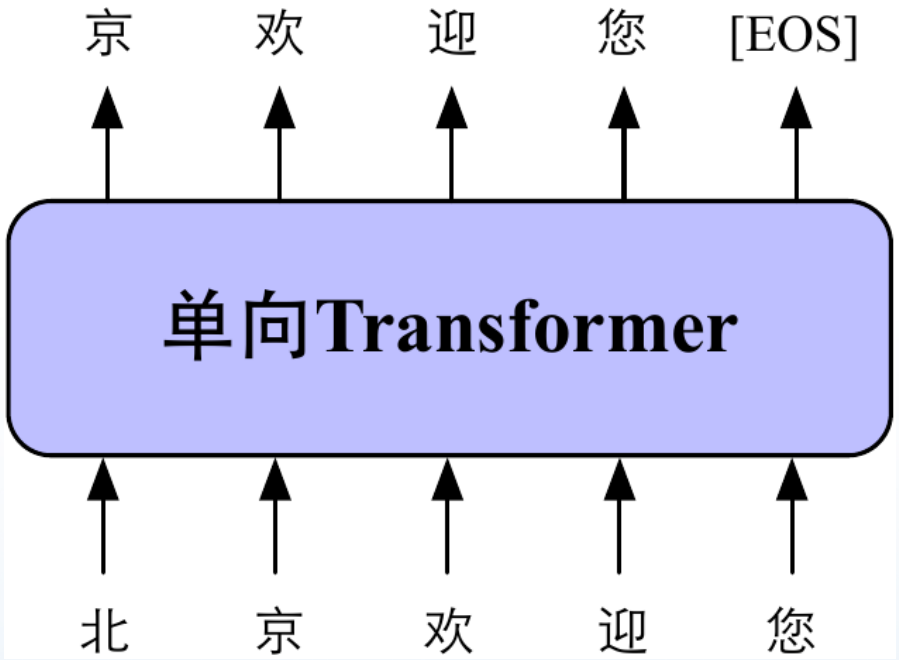


$$V = V^t + V^s + V^p$$

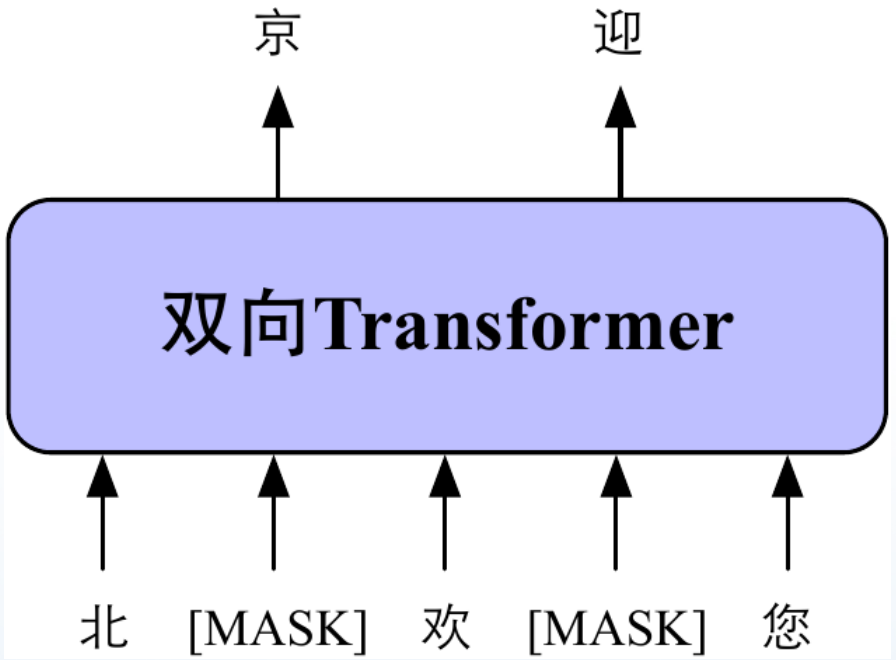
组成	作用
词元向量 V^t	WordPiece词表（约3万），[CLS]开头，[SEP]分隔
块向量 V^s	第一段编码为0，第二段编码为1
位置向量 V^p	可学习的绝对位置，最长512

三种向量维度相同，直接相加。超长截断，不足补[PAD]。

为什么需要掩码：双向建模的信息泄露



单向语言模型：只用左侧上下文



掩码语言模型：遮住后用两侧还原

直接双向预测会泄露答案；先遮住再还原，才能真正利用两侧上下文。

ELMo用两个独立的单向模型拼接；BERT用掩码在同一个深层模型中实现双向。

掩码语言模型：15%与80/10/10

处理方式	训练样本（原文：The man went to the store to buy some milk.）
80%：替换为[MASK]	The man went to the **[MASK]** to buy some milk.
10%：替换为随机词	The man went to the **apple** to buy some milk.
10%：保持原词	The man went to the **store** to buy some milk.

$P_i = \text{Softmax}(h_i^m W^{tT} + b^o)$，只在被选中的位置计算交叉熵

选多少 15%的词元；n = 20时，k = $\lfloor 20 \times 15\% \rfloor = 3$

为何不全用[MASK] 下游没有[MASK]，混入随机词和原词，缓解预训练—精调不一致

“掩码”是广义的：三种处理方式按概率三选一，而不只是替换为[MASK]。

PyTorch实现：生成MLM训练样本

```
def mask_tokens(ids, special, mask_id, vocab_size, p=0.15):
    labels = ids.clone()
    prob = torch.full(ids.shape, p)
    prob[special] = 0.0                          # 不选[CLS]、[SEP]、[PAD]
    picked = torch.bernoulli(prob).bool()
    labels[~picked] = -100                       # 只在被选中的位置计算损失
    to_mask = torch.bernoulli(torch.full(ids.shape, 0.8)).bool() & picked
    ids = ids.masked_fill(to_mask, mask_id)                # 80%: [MASK]
    to_rand = torch.bernoulli(torch.full(ids.shape, 0.5)).bool() & picked & ~to_mask
    ids[to_rand] = torch.randint(vocab_size, ids.shape)[to_rand]   # 10%: 随机词
    return ids, labels                           # 其余10%: 保持原词
```

实测（2000×128个词元）：选中14.99%，其中[MASK] 80.1%、随机词9.9%、原词10.0%。

下一个句子预测（NSP）

文本段	正样本（IsNext）	负样本（NotNext）
第一段文本	The man went to the store.	The man went to the store.
第二段文本	He bought a gallon of milk.	Penguins are flightless.

$$X = [CLS]\ x^{(1)}\ [SEP]\ x^{(2)}\ [SEP], \quad P = Softmax(h_0 W^p + b^o)$$

样本构造　正负各50%，负样本的第二段从语料中随机抽取

作用　为阅读理解、文本蕴含等句对任务学习两段文本的关系

后续发现　任务过于简单，RoBERTa去掉NSP后效果不降反升

h_0是[CLS]位的最后一层表示，句对分类任务精调时也用它。

整词掩码（WWM）

英文示例	样本
原始句子	the man jumped up , put his basket on phil ##am ##mon ' s head
原始掩码	[M] man [M] up , put his [M] on phil [M] ##mon ' s head
整词掩码	the man [M] up , put his basket on [M] [M] [M] ' s head

中文示例	样本
中文分词	使用 语言 模型 来 预测 下 一 个 词 的 概率 。
原始掩码	使 [M] 语 言 [M] 型 来 [M] 测 下 一 [M] 词 的 概 率 。
整词掩码	使 用 语 言 [M] [M] 来 [M] [M] 下 一 个 词 的 概 率 。

中文需先分词确定词边界。整词中各子词的80/10/10处理方式仍分别抽取。

N-gram掩码与三种策略对比

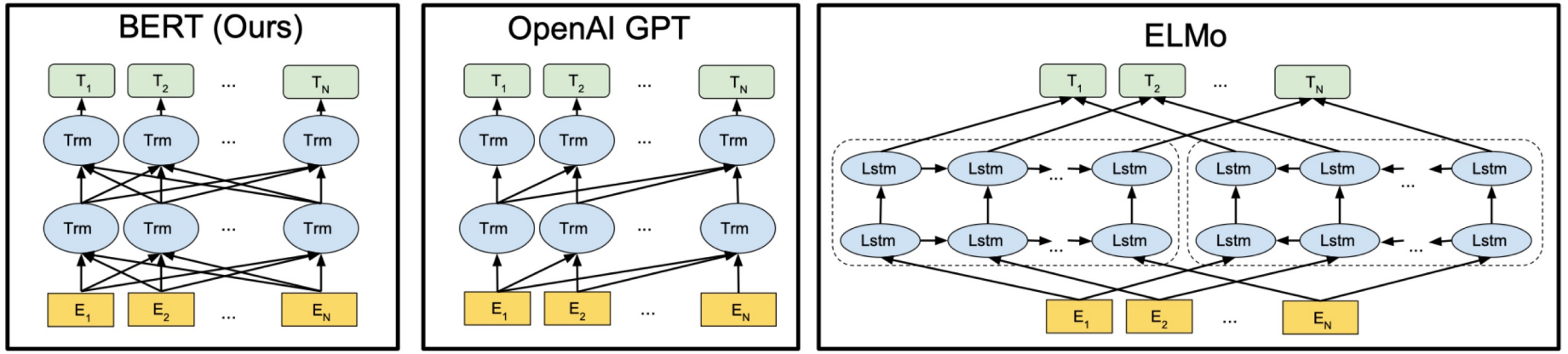
比较项目	掩码语言模型	整词掩码	N-gram掩码
最小掩码单位（英文）	WordPiece子词	WordPiece子词	WordPiece子词
最小掩码单位（中文）	字	字	字
最大掩码单位（英文）	WordPiece子词	词	多个子词
最大掩码单位（中文）	字	词	多个字

N-gram长度　从1-gram到4-gram的选择概率分别为40%、30%、20%、10%

预测难度　N-gram掩码 > 整词掩码 > 掩码语言模型

对下游透明　只影响预训练时选哪些位置，精调代码无须任何改动

BERT与GPT、ELMo、Word2vec



对比项目	BERT	GPT	ELMo	Word2vec
基本结构	Transformer	Transformer	Bi-LSTM	MLP
训练任务	MLM / NSP	语言模型	双向语言模型	Skip-gram / CBOW
建模方向	双向	单向	双向（拼接）	双向窗口
静态 / 动态	动态	动态	动态	静态

上图：BERT（左）、GPT（中）、ELMo（右）的连接方式。Devlin等，2019。

RoBERTa：动态掩码与舍弃NSP

静态掩码 vs. 动态掩码

掩码方法	SQuAD 2.0（F1）	MNLI-m（%）	SST-2（%）
静态掩码	78.3	84.3	92.5
动态掩码	78.7	84.0	92.9

是否需要NSP

输入设置	SQuAD 1.1	MNLI-m	SST-2	RACE
文本对输入 + NSP	90.4	84.0	92.9	64.2
句子对输入 + NSP	88.7	82.9	92.1	63.0
跨文档整句（无NSP）	90.4	84.7	92.5	64.8
文档内整句（无NSP）	90.6	84.7	92.7	65.6

最终采用跨文档整句输入：效果接近最好，且批次大小固定，便于大规模训练。

RoBERTa：更多数据、更大批次、更大词表

数据	类型	大小
BookCorpus + 维基	故事、百科	16GB
CC-News	新闻	76GB
OpenWebText	网页	38GB
Stories	故事	31GB

批次	步数	PPL	MNLI-m	SST-2
256	1M	3.99	84.7	92.5
2K	125K	3.68	85.2	93.1
8K	31K	3.77	84.4	93.2
8K	125K	3.50	85.8	94.1

词表 3万WordPiece → 5万字节级BPE（同GPT-2），不再出现[UNK]

最终设置 约160GB数据、批次8K、训练50万步，去掉NSP、使用动态掩码

Liu等，RoBERTa，2019。批次实验中PPL为开发集困惑度。

ALBERT：词向量因式分解

$$O(V \times H) \rightarrow O(V \times E + E \times H), \quad E \ll H$$

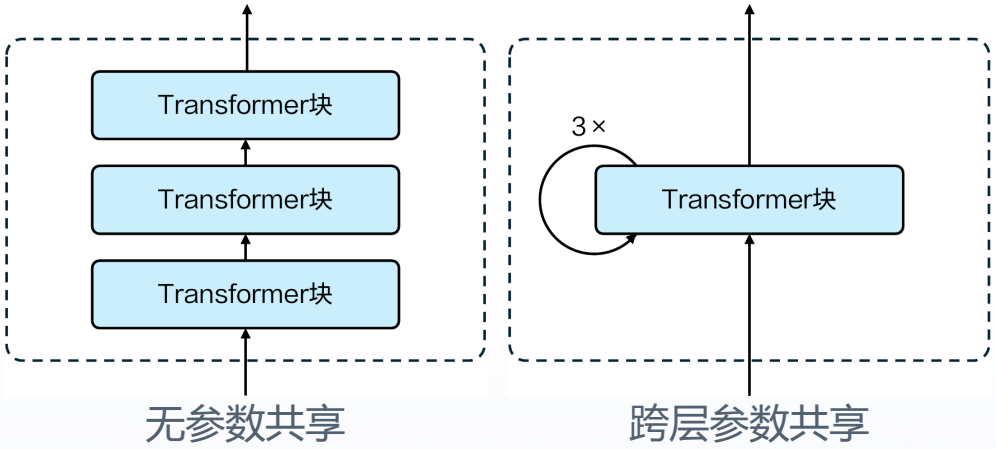
方案	词向量部分参数量（V = 30000，H = 1024）	结果
BERT（E = H）	30,000 × 1,024	30,720,000
ALBERT（E = 128）	30,000 × 128 + 128 × 1,024	3,971,072

词向量参数降至约1/8

动机　　词向量只需上下文无关的表示，模型容量应放在隐含层上

Lan等，ALBERT，ICLR 2020。E为词向量维度，H为隐含层维度，V为词表大小。

ALBERT：跨层参数共享与句子顺序预测



3层主体	参数存储	激活	块计算
不共享	3×	3×	3×
共享	**1×**	3×	3×

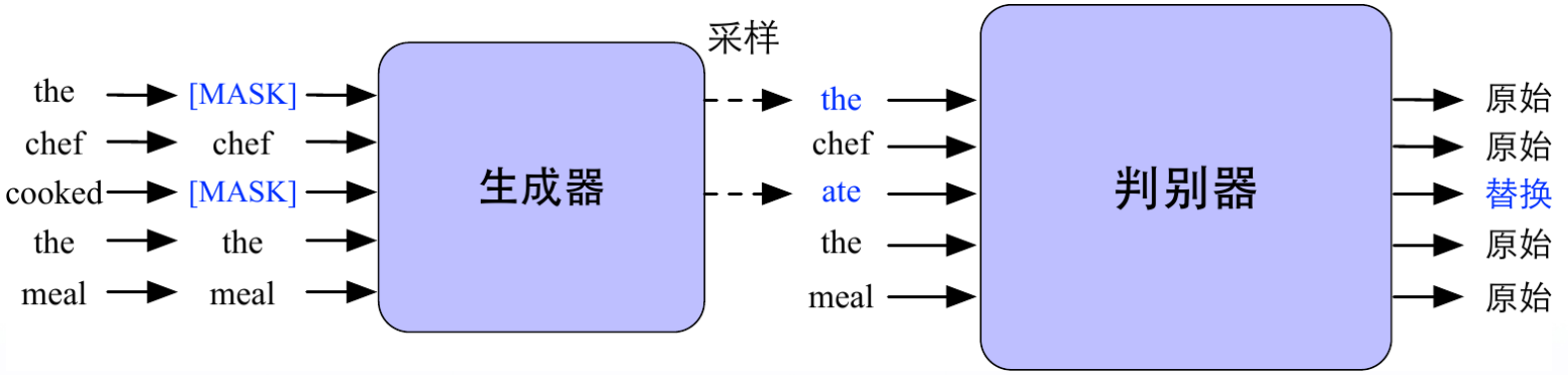
只减少参数与相关存储，不减少计算量。

SOP正例　　两段连续文本，保持原有顺序

SOP负例　　同样两段文本，交换顺序：同主题，只能靠连贯性判断

句子顺序预测（SOP）取代NSP，MacBERT也采用了它。

ELECTRA：替换词检测



步骤	1	2	3	4	5
原句	the	chef	cooked	the	meal
掩码	[MASK]	chef	[MASK]	the	meal
生成器采样	the	chef	**ate**	the	meal
判别器标签	0	0	**1**	0	0

效率　　判别器在全部位置上学习；MLM只在约15%的位置上计算损失

生成器约为判别器的1/4～1/2（base为1/3），预训练后丢弃，只用判别器精调。

MacBERT：用相似词代替[MASK]

掩码方式	样本
原始句子	使用语言模型来预测下一个词的概率。
原始掩码	使 用 语 言 [M] 型 来 [M] 测 下 一 个 词 的 概 率 。
整词掩码	使 用 语 言 [M] [M] 来 [M] [M] 下 一 个 词 的 概 率 。
N-gram掩码	使 用 [M] [M] [M] [M] 来 [M] [M] 下 一 个 词 的 概 率 。
纠错型掩码（Mac）	使 用 **语 法 建 模** 来 **预 见** 下 一 个 词 的 **几 率** 。

其余设置　15%的位置：80%换相似词，10%换随机词，10%保持原词；用SOP代替NSP

Cui等，Revisiting Pre-Trained Models for Chinese NLP，Findings of EMNLP 2020。

Encoder-only模型对比

模型	类型	分词	预训练任务	数据规模
BERT	自编码	WordPiece	MLM + NSP	约16GB
XLNet	自回归	SentencePiece	排列语言模型	约126GB
RoBERTa	自编码	**字节级BPE**	MLM	约160GB
ALBERT	自编码	SentencePiece	MLM + SOP	约16GB
ELECTRA	自编码	WordPiece	生成器 + 判别器	约126GB
MacBERT	自编码	WordPiece	Mac + SOP	约20GB

改进主要集中在三处：掩码怎么做、数据与训练怎么扩、参数与计算怎么省。

T5的预训练：片段破坏

序列	内容
原文本	Thank you for inviting me to your party last week .
输入序列（随机丢弃15%）	Thank you <X> me to your party <Y> week .
目标序列（只还原丢弃片段）	<X> for inviting <Y> last <Z>

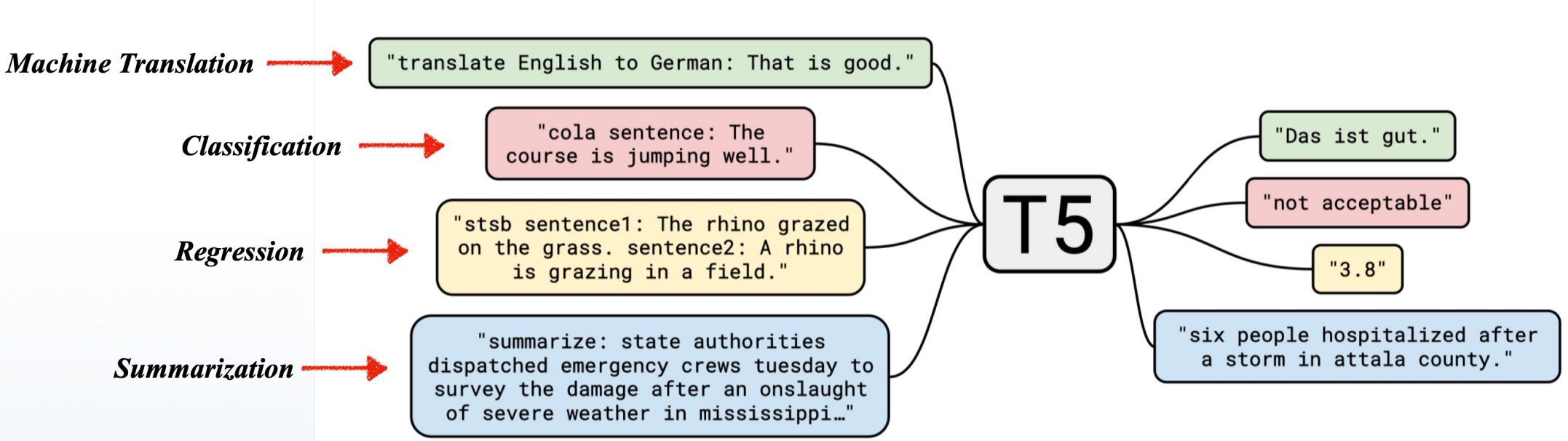
哨兵词元 每个被删片段用一个不同的哨兵表示，<Z>标记目标序列结束

相比BERT 一个哨兵可对应多个词，由解码器自回归生成

多任务训练 也可混入GLUE、SQuAD、翻译等有监督任务

目标序列只包含被删除的片段，比重构整句更短、更省计算。

T5：一切任务都是“文本到文本”

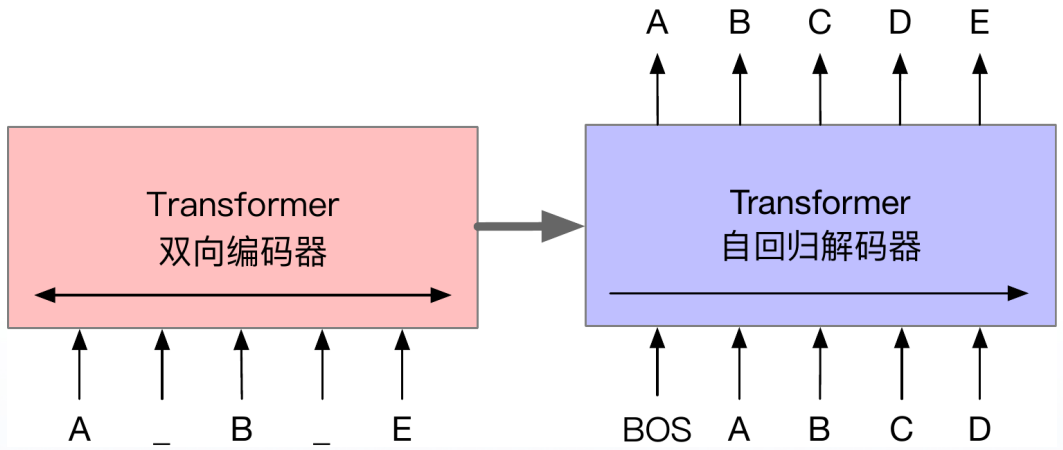


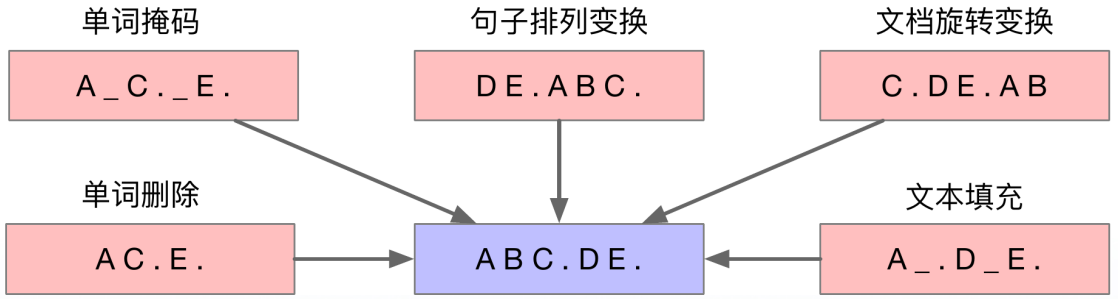
统一接口 用任务前缀注入任务信息，输入和输出都是文本

规模与数据 最大约110亿参数；C4语料约750GB英文网页文本

Raffel等，Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer，JMLR 2020。

BART：去噪自编码器





结构 双向编码器读取带噪文本，自回归解码器重构原文

五种噪声 单词掩码、单词删除、句子排列、文档旋转、文本填充

最好的组合 文本填充（片段长度 ~ Poisson(λ = 3)）再加句子排列

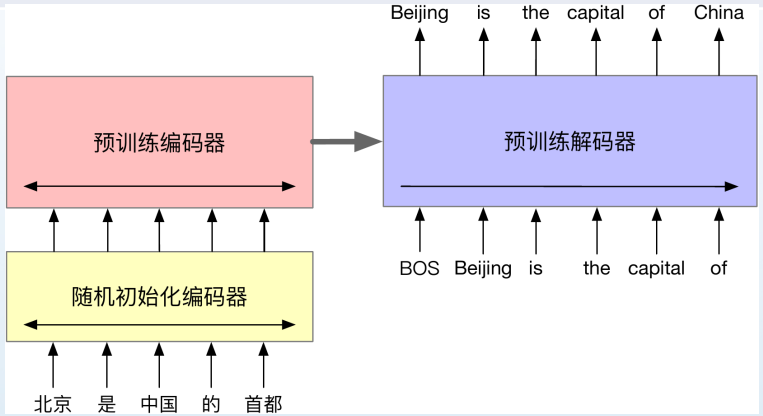
Lewis等，BART，ACL 2020。

BART用于下游任务

任务	编码器输入	用于预测的表示
序列分类	原文本（解码器同输入）	解码器末尾特殊词元的隐状态
序列标注	原文本（解码器同输入）	解码器每个位置的隐状态
文本生成	作为条件的文本	解码器自回归生成目标文本
机器翻译	源语言（经新加的小编码器）	解码器生成目标语言

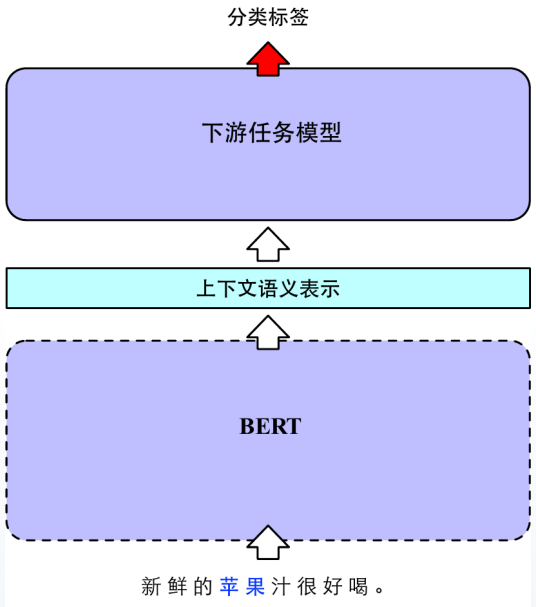
翻译的两步训练

先只训练新加的源语言编码器等少数参数，再少量迭代精调全部参数

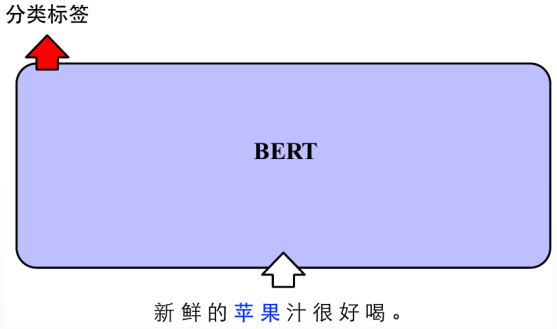


右图：机器翻译中，随机初始化的编码器把源语言映射到BART的输入空间。

两种使用方式：特征提取与模型精调



特征提取（BERT冻结）

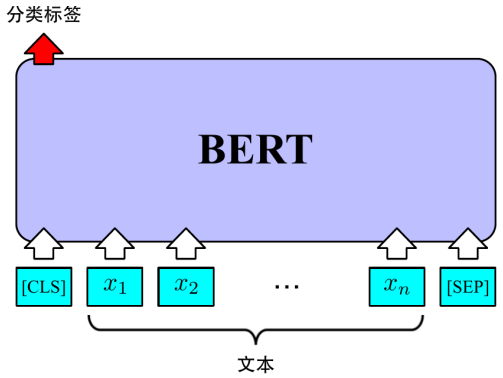


模型精调（BERT参与训练）

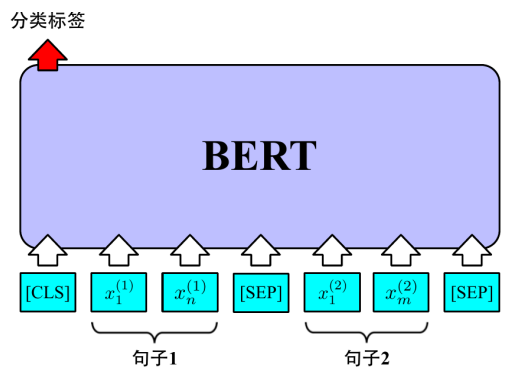
比较	特征提取	模型精调
BERT参数	冻结	一起更新
训练开销	小	大
任务适配	靠下游模型	模型自身适配
典型效果	较差	**明显更好**

本章以下均以模型精调为例。

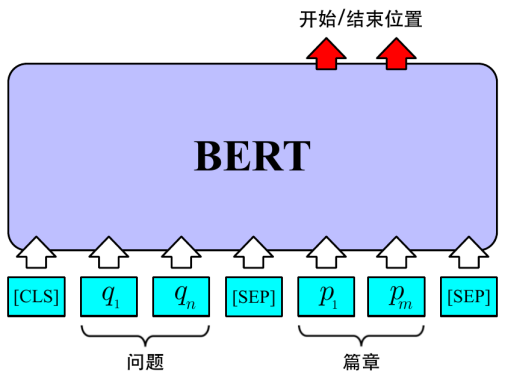
四类NLU任务的模型结构



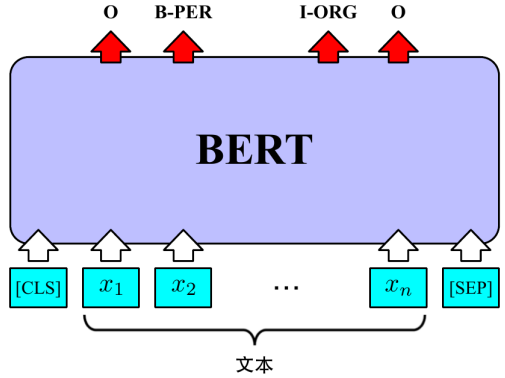
单句分类（SST-2）



句对分类（RTE）



阅读理解（SQuAD）



序列标注（CoNLL-2003）

同一个预训练模型，只换输入的拼接方式和输出层。

图来自教材第7章“预训练模型的任务微调：NLU类”。

输入格式与输出层

任务	输入序列	输出层	损失
单句分类	[CLS] x [SEP]	h_0 → K类Softmax	交叉熵
句对分类	[CLS] $x^{(1)}$ [SEP] $x^{(2)}$ [SEP]	h_0 → K类Softmax	交叉熵
阅读理解	[CLS] 问题 [SEP] 篇章 [SEP]	每个位置 → 起点、终点分数	$(\mathcal{L}^s + \mathcal{L}^e) / 2$
序列标注	[CLS] x [SEP]	每个位置 → K个BIO标签	逐位置交叉熵

$$\mathbf{P} = \mathrm{Softmax}(h_0 W^o + b^o), \quad W^o \in \mathbb{R}^{d \times K}$$

序列标注也可在输出层之上加条件随机场（CRF）。

抽取式阅读理解：预测起止位置

【篇章】
……学校于1996年进入国家“211工程”首批重点建设高校，**1999年**被确定为国家首批“985工程”重点建设的9所大学之一……
【问题】
哈尔滨工业大学在哪一年入选了国家首批“985工程”？
【答案】1999年

$$P^s = \mathrm{Softmax}(h W^s + b^s)$$

$$P^e = \mathrm{Softmax}(h W^e + b^e)$$

$$\mathcal{L} = (\mathcal{L}^s + \mathcal{L}^e) / 2$$

解码：起点、终点各取Top-k，
按$P^s_i \cdot P^e_j$从大到小，
选第一个满足“起点 ≤ 终点”的片段。

输入顺序　　问题在前、篇章在后：超长时只截断篇章，可滑窗切片多次预测

数据集：SQuAD（英文）、CMRC 2018（中文）。答案是篇章中的一个连续片段。

序列标注：子词与标签对齐

	John	Smith	has	never	been	to	Harbin	
原始标签	B-PER	I-PER	O	O	O	O	B-LOC	
切分后	John	Smith	has	never	been	to	**Ha**	**##rbin**
对齐后标签	B-PER	I-PER	O	O	O	O	**B-LOC**	**B-LOC**

BIO B：实体开始；I：实体内部（Inside）；O：非实体

对齐策略 教材：子词继承原标签；常见做法：只在首个子词计算损失，其余设为−100

评价时以词或实体为单位（如seqeval计算F1），不按子词计算。

代码实现：用transformers精调BERT

```
from datasets import load_dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
import evaluate, numpy as np

data = load_dataset("nyu-mll/glue", "sst2")
tok = AutoTokenizer.from_pretrained("bert-base-cased")
data = data.map(lambda b: tok(b["sentence"], truncation=True), batched=True)
model = AutoModelForSequenceClassification.from_pretrained("bert-base-cased", num_labels=2)
acc = evaluate.load("accuracy")
def metrics(p):
    return acc.compute(predictions=np.argmax(p.predictions, -1), references=p.label_ids)
args = TrainingArguments("ft-sst2", eval_strategy="epoch", learning_rate=2e-5,
                         per_device_train_batch_size=16, num_train_epochs=2)
trainer = Trainer(model=model, args=args, train_dataset=data["train"],
                  eval_dataset=data["validation"], processing_class=tok,
                  compute_metrics=metrics)
trainer.train()
```

已用transformers 5.19实测接口。教材中的load_metric、evaluation_strategy在新版中已移除。

生成类任务：GPT-2续写与T5翻译

任务	模型	训练数据	评价
文本生成	GPT-2（Decoder-only）	WikiText-2	困惑度 PPL = exp(平均交叉熵)
机器翻译	mT5-small（Encoder-Decoder）	IWSLT2017 中英	BLEU，并人工检查译文

共同点 都按“给定前文/源文，逐词生成”的方式训练，损失为目标词的交叉熵

区别 GPT-2把条件与目标拼成一条序列；T5用编码器读源文、解码器生成译文

生成时可用贪心、集束搜索或Top-k/Top-p采样，详见第5章。

研究进展：结构组件的变化

组件	2018年（BERT、GPT）	当前主流大模型（如Llama 3、Qwen3）
位置编码	可学习的绝对位置向量	旋转位置编码（RoPE）
归一化	后置LayerNorm（GPT-2起前置）	前置RMSNorm
前馈激活	GELU	SwiGLU
注意力	多头注意力	分组查询注意力（GQA），减小KV缓存
词表规模	3万～5万	约13万～15万
上下文长度	512～1024	3.2万～12.8万
预训练数据	约16GB～40GB文本	15万亿～36万亿词元

数值以Llama 3与Qwen3为例。Grattafiori等，2024；Qwen团队，2025。

研究进展：编码器的新生

项目	BERT-base（2018）	ModernBERT-base（2024）
参数量 / 层数	约1.1亿 / 12层	约1.49亿 / 22层
最大长度	512	8192
训练数据	约33亿词	约2万亿词元（含代码）
位置编码	绝对位置向量	RoPE
注意力与其他	全局注意力	局部—全局交替注意力、GeGLU、去补齐

后续工作　mmBERT（2025）：1800多种语言；Ettin（2025）：同数据同配方的成对编码器与解码器

编码器仍是检索、分类和嵌入模型的主力。Warner等，2024；Marone等，2025；Weller等，2025。

研究进展：为什么Decoder-only成为主流

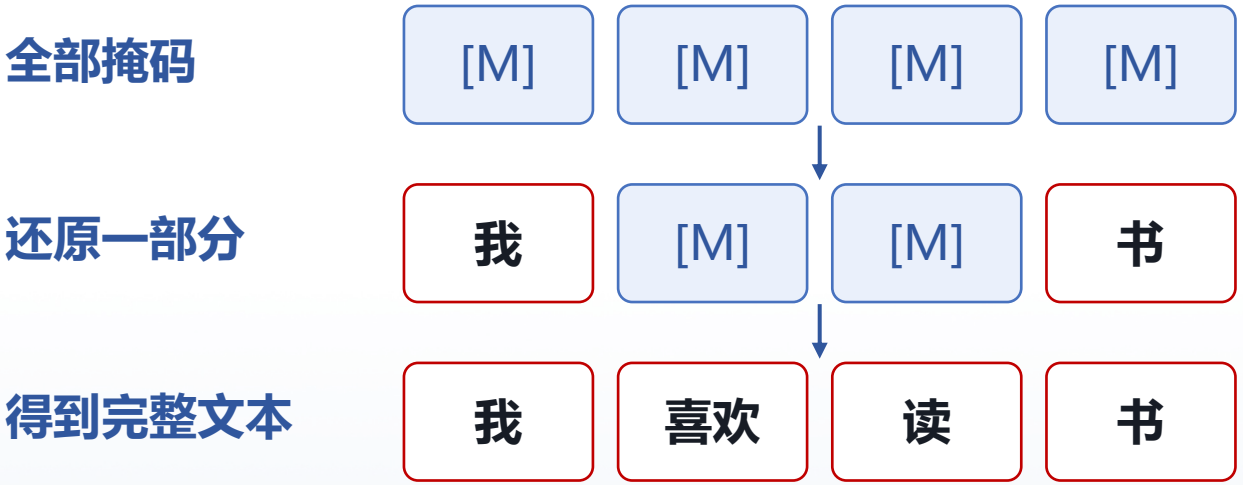
统一接口	任何任务都能写成“给定前文，生成答案”，与提示、语境学习天然契合
训练信号	每个位置都有预测目标；MLM只在约15%的位置计算损失
规模与推理	随规模增长出现小样本能力；生成时可复用KV缓存

但没有绝对的优劣

- 同等数据与配方下：编码器更擅长分类与检索，解码器更擅长生成（Ettin，2025）
- 仅自监督预训练：因果解码器零样本最好；加多任务精调后：非因果 + MLM最好（Wang等，2022）
- T5Gemma（2025）：用解码器权重初始化编码器—解码器模型

选结构要看任务：生成与通用对话多用解码器，检索与分类常用编码器。

研究进展：从掩码语言模型到扩散语言模型



BERT：固定遮住15%，只用于学习表示

掩码扩散：掩码比例t在0～1间随机，从全掩码逐步还原即可生成文本

LLaDA（2025） 8B模型从头训练，作者报告在语境学习上与LLaMA3 8B相当

LLaDA2.0（2025） 由自回归模型转换而来，扩展到1000亿参数的MoE

仍属活跃研究：推理效率与生成质量需按任务比较。

本章小结

三种结构	注意力可见范围决定了擅长理解还是擅长生成
Decoder-only	GPT：因果语言模型预训练 → 精调 → 语境学习
Encoder-only	BERT：MLM + NSP；改进集中在掩码、数据与效率
Encoder-Decoder	T5、BART：文本到文本，去噪预训练
下游应用	同一模型，只换输入格式与输出层

后续课程：语言模型的微调。

谢 谢!

理解语言，认知社会

以中文技术，助民族复兴



长按二维码，关注哈工大SCIR

微信号：HIT_SCIR